PERCORSO INTENSIVO DI AI SOFTWARE ENGINEERING

# Lezione 9 Skills, subagent, worktree, Superpowers e Ralph loops

**Dal comportamento corretto del software alla progettazione del processo con cui gli agenti lo producono e lo verificano.**

> Obiettivo: imparare a usare procedure riutilizzabili, agenti delegati, checkout Git separati, orchestrazione su task graph e loop iterativi senza perdere la responsabilità sul dominio e sulle prove di correttezza.

Tempo di studio: 35–45 minuti, compreso il laboratorio di 15 minuti. Prerequisiti: Lezioni 1–8 (domain discovery, requisiti, specification, TDD, architecture boundaries, contract/property-based testing, context engineering).

## Mappa della lezione

1. Tassonomia e responsabilità · 2. Skills, command e hooks · 3. Subagent e contratti di delega · 4. Git worktree · 5. Task graph e parallelismo · 6. Superpowers e Matt Pocock · 7. Ralph loops · 8. Caso Quote · 9. Failure mode · 10. Laboratorio e criteri di successo.

# 1. Dalla correttezza del codice alla correttezza del processo

Finora abbiamo studiato come ridurre la distanza tra un bisogno reale e il comportamento del software: Domain → Requirements → Spec → Test → Agent → Verify. Ora aggiungiamo una domanda: come rendere affidabile il processo quando l’implementazione viene distribuita fra più incarichi, sessioni e agenti?

Il moltiplicarsi degli agenti aumenta sia la capacità di esecuzione sia le opportunità di introdurre incomprensioni. Un frontend e un backend prodotti da agenti diversi possono avere test unitari verdi e un contratto incompatibile. Due agenti possono non avere conflitti Git e condividere una business policy inventata. Di conseguenza l’autonomia operativa richiede una struttura di governance del lavoro, non soltanto prompt più lunghi.

> Principio guida: scalare l’implementazione senza scalare la capacità di verifica significa aumentare la velocità con cui si possono produrre errori.

## Classificazione essenziale

| Elemento | Categoria | Problema risolto |
|---|---|---|
| Skill | Artefatto operativo | Procedura riutilizzabile e recuperabile |
| Command | Meccanismo di invocazione | Avvio esplicito di una procedura |
| Hook | Meccanismo automatico | Controlli collegati a eventi |
| Subagent | Ruolo di esecuzione | Delega e isolamento del contesto |
| Orchestrator | Componente/ruolo | Dipendenze, assegnazione, integrazione |
| Git worktree | Funzionalità Git | Working directory distinte sullo stesso repo |
| Superpowers | Framework workflow | Disciplina di planning, TDD e review |
| Ralph loop | Tecnica iterativa | Ripetizione con feedback e progresso persistente |

Queste parole non devono essere usate come sinonimi. Una skill non dispone, di per sé, di capacità operative: sono gli strumenti e i permessi dell’harness a fornirle. Un subagent non è necessariamente un processo autonomo permanente. Un worktree non è una sandbox. Un Ralph loop non è necessariamente multi-agente.

# 2. Skills: conoscenza procedurale on-demand

Una skill codifica il «come si fa una classe di lavori». Risolve il problema di dover ripetere a ogni nuova sessione le stesse istruzioni: individuare una specifica, lavorare test-first, non inventare requisiti, documentare l'esito. Nell'ecosistema Claude Code, una skill è generalmente una cartella con un SKILL.md composto da frontmatter YAML e istruzioni Markdown; può essere chiamata esplicitamente o caricata dal modello quando pertinente. La posizione concreta varia per harness.

```
---
name: api-boundary-review
description: Review changes to external API contracts.
---
# Goal
Identify consumer/provider incompatibilities.
# Procedure
1. Read the authoritative contract.
2. Inspect actual consumer requests.
3. Execute contract tests.
4. Report breaking changes with evidence.
# Limits
Do not invent policies; do not change the API silently.
```

La skill non deve trasformarsi in una specifica di business nascosta. Scrivere in una skill «applica uno sconto del 10% ai clienti Premium» sarebbe scorretto se la decisione non proviene da una fonte autorizzata. La skill può invece richiedere di leggere la regola presso la fonte pertinente.

## Separazione fra contesto e procedura

AGENTS.md / CLAUDE.md: informazioni di progetto che devono orientare il lavoro, convenzioni, confini architetturali, link ai documenti e alle ADR. SKILL.md: procedimento specialistico, trigger, passi, strumenti, criteri di uscita. Le procedure lunghe caricate soltanto quando servono aiutano a preservare la capacità della context window.

## Skill, command, hook e tool

| Concetto | Esempio | Nota |
|---|---|---|
| Skill | Review di compatibilità API | Il modello interpreta una procedura |
| Command | /review-api | Punto di ingresso esplicito |
| Hook | Lint dopo la modifica di un file | Si attiva su un evento |
| Tool | Read file / Run test / API call | Fornisce una capacità operativa |

Un divieto critico non dovrebbe dipendere soltanto dal testo di una skill. Se vogliamo impedire modifiche a file contenenti segreti, aggiungiamo autorizzazioni, sandbox, controlli preventivi o policy esterne. Le istruzioni del modello sono informative; un controllo eseguibile può essere applicato in modo deterministico, nei limiti dell'ambiente in cui opera.

### Quando usarla, quando no

Usare skills per procedure ripetitive, cognitivamente articolate o condivise fra sessioni. Evitare di introdurle per piccoli cambiamenti una tantum che si descrivono adeguatamente in due righe. Alternative: script, checklist, regole CI, istruzioni contestuali, documentazione del repository.

## 3. Subagent: contratti di delega e context isolation

Un subagent esegue una responsabilità specifica. Può essere un repository explorer, un reviewer di contratti o un implementer. A seconda dell'harness parte con contesto separato, prompt dedicato e limiti di strumenti; alcune modalità consentono un fork del contesto principale. L'isolamento riduce l'inquinamento del contesto centrale: ad esempio 20.000 righe di log possono essere analizzate dal subagent, che restituisce soltanto file rilevanti, risultati e questioni aperte.

La compressione del risultato è però un rischio: il subagent potrebbe omettere una prova o riassumere male un requisito. Per questo servono riferimenti ai file, esiti dei test e una distinzione esplicita fra KNOWN, ASSUMED, UNKNOWN e DECIDED.

### Un task brief da dare a un subagent

```
task_id: QUOTE-API-12
goal: Check QuoteAdapter against the agreed API contract.
scope: [src/quote, tests/quote]
authoritative_sources: [docs/quote-contract.md]
allowed_changes: none
non_goals: [change prices, deploy, change discount rules]
required_evidence: [file paths, test commands, failures]
completion: Report supported findings and open questions.
```

Il brief è un «contratto di delega», diverso dal contratto HTTP della Lezione 7. Deve precisare obiettivo, ambito, fonti, autorizzazioni, non-obiettivi, artefatti richiesti ed evidenze di verifica. Una buona delega minimizza il numero di decisioni arbitrarie che l'agente deve prendere.

### Quando non delegare

Evitare il subagent per compiti minuscoli, strettamente legati all'attività principale o dove il costo di trasferimento del contesto supera il beneficio. Non assegnare a un implementer decisioni di business ancora UNKNOWN. La review dovrebbe essere sufficientemente indipendente: non è affidabile un revisore che riceve soltanto la spiegazione dell'implementatore e la assume vera.

## 4. Git worktree: isolamento Git, non isolamento totale

Un repository Git può avere un working tree principale e vari working tree collegati. I worktree condividono il repository sottostante, ma hanno directory, indici e checkout separati. Questo consente a più agenti di lavorare su branch distinti senza sovrascrivere direttamente gli stessi file della working directory.

```
git status
git worktree add -b feature/quote-api ../quote-api main
git worktree add -b feature/quote-ui ../quote-ui main
git worktree list
# Solo dopo aver salvato e integrato il lavoro:
git worktree remove ../quote-api
git worktree remove ../quote-ui
```

I comandi assumono che il repository sia pulito e che i branch non esistano già. Di norma Git impedisce di effettuare un checkout dello stesso branch in più worktree contemporaneamente. Verificare lo stato prima di rimuovere; non usare opzioni di rimozione forzata senza aver controllato modifiche e file non tracciati.

### Il falso mito della sandbox

Git worktree non blocca automaticamente l'accesso ad altri percorsi o ai servizi esterni. Processi dei due agenti possono condividere porte, database, Docker daemon, cloud account, cache e segreti. Per ottenere isolamento di sicurezza occorrono ulteriori meccanismi: container, credenziali con privilegio minimo, reti dedicate, perimetri di filesystem e configurazioni dei permessi dell'harness.

Anche worktree separati non risolvono incompatibilità logiche: due agenti possono implementare campi JSON con nomi diversi senza produrre alcun conflitto Git. Il contract testing protegge il boundary; il worktree protegge il checkout.

## 5. Task graph: la condizione del parallelismo utile

Il Directed Acyclic Graph (DAG) descrive task e dipendenze, rendendo visibile la cosiddetta ready frontier: i task i cui predecessori sono completati e verificati. Soltanto i nodi pronti dovrebbero essere considerati per l'esecuzione, eventualmente in parallelo.

```
T1: Define and approve Quote API contract
              |
       +------+------+
       |             |
T2: Pricing API   T3: Quote UI
       |             |
       +------+------+
              |
T4: Contract + integration + acceptance verification
```

Dopo T1, T2 e T3 possono procedere in parallelo; T4 richiede entrambi. Se T1 è UNKNOWN, il parallelismo deve fermarsi: non ha senso accelerare l'esecuzione di due interpretazioni incompatibili. Il costo del progetto non dipende soltanto dalla somma delle durate delle attività, ma dal percorso critico, dal coordinamento, dall'integrazione e dalla rilavorazione.

> Regola pratica: prima si progettano i confini dei ticket e le loro dipendenze; solo dopo si decide quanti agenti assegnare.

Un orchestrator coordina preparazione, avvio, monitoraggio, merge e revisione. Non diventa automaticamente l'autorità sulle policy di dominio. Il criterio di scelta dei task «pronti» deriva da dipendenze e prove, non dall'improvvisazione del modello.

# 6. Superpowers: disciplina riutilizzabile per il ciclo di sviluppo

Superpowers è un framework di workflow open source sviluppato da Jesse Vincent e collaboratori. È organizzato come collezione di skill componibili per coding agent. La documentazione ufficiale presenta un flusso in cui il lavoro viene discusso prima di codificare, trasformato in un design e in un piano, implementato con TDD e review, infine verificato prima del completamento.

```
Brainstorm / design
        -> Approve
        -> Write implementation plan
        -> Execute (optionally through subagents/worktrees)
        -> TDD / code review
        -> Verification before completion
```

Sono particolarmente rilevanti le skill brainstorming, writing-plans, using-git-worktrees, subagent-driven-development, test-driven-development e verification-before-completion. Sono meccanismi operativi. Non sostituiscono la necessità di requisiti derivati dal dominio, di contract test o di test oracles indipendenti.

## Matt Pocock e Superpowers: dove si incontrano

| Fase | Matt Pocock | Superpowers |
|---|---|---|
| Discovery | grill-with-docs, domain-modeling | brainstorming |
| Decisioni | Glossario e ADR | design document + approval |
| Spec | to-spec | design approvato / plan |
| Decomposizione | to-tickets / task graph | writing-plans |
| Implementazione | implement / implement-spec | executing-plans / subagent-driven-development |
| Review | code-review | requesting-code-review / verification |

Questa tabella è un confronto concettuale, non un'affermazione di equivalenza dei singoli strumenti. Matt Pocock pone forte enfasi sul linguaggio condiviso e sulle decisioni esplicite; Superpowers esplicita un processo rigoroso di sviluppo, TDD e verifica. Si possono combinare selettivamente, ma non è necessario sovrapporre due pipeline complete per ogni modifica.

Nota sulle versioni: nel repository attuale di Matt Pocock il documento di glossario è indicato come GLOSSARY.md. In vecchi esempi compariva CONTEXT.md. La convenzione di file evolve; il principio del durable context non cambia.

# 7. Ralph loop: iterare con feedback e memoria esterna

Geoffrey Huntley descrive Ralph come una tecnica semplice di looping su un coding agent: un task alla volta, feedback dalla verifica e progresso persistente negli artefatti del repository. Il punto non è moltiplicare agenti, bensì ridurre il carico della singola sessione e sfruttare ripetizioni guidate da segnali osservabili.

```
SPEC + TASK LIST
       |
       v
   ONE TASK
       |
       v
 AGENT EXECUTION
       |
       v
 TEST / FEEDBACK
       |
       v
 PERSIST PROGRESS ---> NEXT ITERATION
```

Se l'intero contesto della conversazione non viene conservato, assumono particolare importanza la specification, la lista dei ticket, le decisioni, i commit e gli esiti verificabili. Questo si collega direttamente alla Lezione 8: il durable context non è un riassunto vago, ma un insieme di artefatti che permette a una nuova sessione di prendere decisioni coerenti.

## Loop controllato: pseudocodice didattico

```
MAX_ITERATIONS = 8
for iteration in range(MAX_ITERATIONS):
    task = select_next_ready_task()
    if task is None:
        break
    run_agent(task=task, spec="spec.md")
    evidence = run_verification(task)
    save_report(task, evidence)
    if evidence.passed:
        mark_task_complete(task)
    if requires_human_decision(task):
        break
```

Lo pseudocodice non è un orchestrator production-ready. Manca, per esempio, la gestione delle eccezioni, la sicurezza dell'esecuzione, la registrazione di ogni comando, il controllo delle risorse, il retry policy e la sincronizzazione degli artefatti. Serve a esprimere un principio: un loop ingegneristico deve prevedere budget, condizioni di arresto, feedback riproducibile e possibilità di escalation umana.

Un Ralph loop può utilizzare un unico agente; una pipeline multi-agente può usare task paralleli senza alcun loop di tipo Ralph. Le tecniche possono essere combinate, ma non coincidono.

# 8. Caso integrato: Quote Service

Supponiamo di dover realizzare una Quote API. Requisito approvato: il checkout deve richiedere un preventivo e ricevere un totale espresso nelle unità minime della valuta insieme al codice valuta. Nel laboratorio non definiamo sconti o regole fiscali, che restano fuori scope oppure UNKNOWN.

```
POST /quotes
{
  "items": [{ "sku": "PRODUCT-1", "quantity": 2 }]
}

HTTP 200
{
  "total_minor": 2500,
  "currency": "EUR"
}
```

## Autorità e assunzioni

| Voce | Stato | Conseguenza |
|---|---|---|
| response.total_minor | DECIDED | Contract test richiesto |
| response.currency | DECIDED | Verifica di presenza e tipo |
| quantity positiva | DECIDED (esercizio) | Test di validazione |
| politica sconti | UNKNOWN | Non implementarla autonomamente |
| fiscalità | OUT OF SCOPE | Non aggiungere regole fiscali |
| persistenza preventivi | OUT OF SCOPE | Non introdurre database solo per ipotesi |

## La porta applicativa nel frontend

```
type Quote = { total_minor: number; currency: string };

interface QuoteProvider {
  getQuote(items: Array<{
    sku: string;
    quantity: number;
  }>): Promise<Quote>;
}
```

L’HTTP adapter implementa la porta. Un test unitario con fake provider verifica la logica UI senza rete. Il contract test verifica che la richiesta reale del consumer e la risposta del provider siano compatibili. Un acceptance test verifica lo scenario concordato dall’utente. Tutti questi controlli coprono rischi diversi.

### Unit test illustrativi

```
# Python, esempio di calcolo semplificato
items = [{"unit_price_minor": 1250, "quantity": 2}]
assert calculate_quote(items).total_minor == 2500
```

```
// PHP, equivalente concettuale
$items = [['unit_price_minor' => 1250, 'quantity' => 2]];
$this->assertSame(2500, calculateQuote($items)->totalMinor);
```

La funzione e il modello sono volutamente semplificati: il loro comportamento completo dipende dalla specification. Non si deve dedurre dalle due righe di test una politica di arrotondamento, imposte o sconti che non sia stata approvata.

### Esecuzione delegata e integration gate

Dopo aver approvato T1 (contratto), l'orchestrator prepara un worktree backend e uno frontend. Ogni implementer riceve il task e le fonti autorizzate. T2 e T3 possono procedere in parallelo. L'integration gate esegue test del backend e frontend, contract verification, acceptance test e static analysis. Il reviewer confronta il diff con la specification. Solo successivamente interviene l'approvazione umana per il merge/deployment.

## 9. Failure mode da riconoscere

### Task vaghi

«Migliora il progetto» non definisce né ambito né completamento: aumentano modifiche arbitrarie e sovrapposizioni.

### Skill-policy nascosta

Una procedura che prescrive sconti o altri comportamenti non autorizzati sta inventando dominio.

### Worktree scambiato per sandbox

Isolamento del checkout non significa isolamento di credenziali, database e risorse esterne.

### Review circolare

Il reviewer e l'implementer condividono le stesse assunzioni errate e producono falsi segnali verdi.

### Ralph senza stop

Non sono definiti budget, test, arresto o escalation; un loop può generare molte modifiche senza progresso.

### Overengineering del workflow

La macchina di coordinamento costa più della modifica; per interventi piccoli è preferibile un workflow più leggero.

# 10. Laboratorio guidato (15 minuti)

Obiettivo: progettare un piccolo processo agentico per il Quote Service, senza perdere l’autorità sul dominio. Non è necessario creare un sistema di orchestrazione production-ready.

## Fase A: specification

Scrivi spec.md con Goal, Scope, Non-goals, Business rules, UNKNOWN, contratto API e acceptance example. Assicurati che le regole prive di conferma restino tali.

## Fase B: task graph e delega

Disegna T1 → (T2, T3) → T4. Per ogni ticket definisci input, output, dipendenze, file ammessi, verifiche. Prepara un brief per il worker backend e uno per il worker frontend.

## Fase C: due worktree

In un repository Git di esercitazione pulito usa git worktree add per due branch differenti e verifica con git worktree list. Non cancellare worktree con modifiche non salvate.

## Fase D: incompatibilità intenzionale

Cambia la risposta backend da total_minor ad amount_minor senza cambiare il frontend. Identifica quali contract test o integration test devono segnalare la rottura. Spiega perché un test del solo calcolo del totale potrebbe ancora passare.

## Fase E: Ralph loop bounded

Scrivi pseudocodice con MAX_ITERATIONS, select_next_ready_task, run_agent, run_verification, save_report e stop su decisione umana. Separa un esito dichiarato dal modello da una prova realmente eseguita.

> Esito atteso: sai distinguere una skill da un subagent, un worktree da una sandbox, un task graph da una semplice lista di cose da fare e la conclusione dichiarata da una verifica indipendente.

# 11. Checklist di decisione

| Problema | Approccio ragionevole |
|---|---|
| Procedura ripetuta, richiede ragionamento | Skill |
| Controllo che deve scattare su evento | Hook / CI / policy |
| Ricerca ampia che sporca il contesto | Subagent Explorer |
| Due modifiche indipendenti nel medesimo repo | Worktree separati |
| Task con dipendenze complesse | DAG + orchestrator |
| Workflow di sviluppo disciplinato | Superpowers / skills specializzate |
| Molte piccole iterazioni con feedback forte | Loop bounded tipo Ralph |
| Cambio molto piccolo, facilmente verificabile | Singolo agente + test mirati |

# 12. Takeaway e ponte alla Lezione 10

Skill: conservano la disciplina, non decidono il dominio. Subagent: delegano responsabilità con contesto e permessi delimitati. Worktree: isolano i checkout ma non sono sandbox di sicurezza. Task graph: rende esplicite le dipendenze prima del parallelismo. Superpowers: fornisce una disciplina operativa di sviluppo. Ralph loops: strutturano cicli di lavoro e feedback che richiedono arresto e verifica.

> Formula concettuale: Autonomia operativa = delega delimitata + contesto utile + ambiente controllato + evidenze indipendenti + condizioni di arresto.

Nella Lezione 10 collegheremo questa architettura del lavoro a Verification ed Evaluation-Driven Development: come progettare valutazioni indipendenti e come misurare la qualità dei risultati, non soltanto il numero di task completati.

# Autovalutazione: cinque domande con risposte

### 1. Una skill basta a impedire la lettura di file riservati?

No. La skill può descrivere una regola operativa, ma per una restrizione affidabile sono necessari permessi e controlli implementati nell’ambiente o nelle policy dell’harness.

### 2. Posso distribuire in parallelo task con lo stesso requisito UNKNOWN?

Non è prudente se i task dipendono dalla decisione mancante. La decisione va raccolta da una fonte autorevole, poi incorporata in specification e contratti condivisi.

### 3. Due worktree evitano l’incompatibilità fra frontend e backend?

No. Evitano conflitti diretti nella working directory; la compatibilità applicativa va testata con contratti, integrazione e acceptance example.

### 4. In cosa un Ralph loop differisce da cinque subagent paralleli?

Il primo è un modello di iterazione con progresso persistente, potenzialmente con un solo agente; i secondi rappresentano delega e parallelismo. Si possono combinare, ma sono concetti diversi.

### 5. Quando un agente può dichiarare “finito”?

Quando riporta il completamento del proprio task, non necessariamente quando l’intera feature è approvata. Per il completamento del sistema occorrono verifiche eseguite, integrazione, review e, per le decisioni previste, approvazione umana.

## Fonti autorevoli e aggiornate

Git — worktree
https://git-scm.com/docs/git-worktree

Claude Code — Skills
https://code.claude.com/docs/en/skills

Claude Code — Subagents
https://code.claude.com/docs/en/sub-agents

Claude Code — Hooks
https://code.claude.com/docs/en/hooks

Claude Code — Features overview
https://code.claude.com/docs/en/features-overview

Matt Pocock — Skills
https://github.com/mattpocock/skills

Jesse Vincent / obra — Superpowers
https://github.com/obra/superpowers

Geoffrey Huntley — Ralph Wiggum
https://ghuntley.com/ralph/

Geoffrey Huntley — Everything is a Ralph loop
https://ghuntley.com/loop/

Nota: i nomi dei comandi, i file di configurazione e i workflow specifici dei framework possono cambiare con gli aggiornamenti. Conferma sempre la documentazione relativa alla versione effettivamente installata.